OBJECT ORIENTED PROGRAMMING WITH C++
ASSIGNMENT 4 – CO4

Designing Reusable Class Hierarchies
Inheritance, Virtual Bases, Polymorphism & Pointers

Department of Computer Science & Engineering

Submitted by:
Rashmi Naurene
Register Number: 192521357
Semester: [Semester]

Under the Guidance of
Jegedeesh K

Academic Year 2025–26 | Total Marks: 50

INDEX

Task 1 – Class Design Review (Diagnose & Repair) ..... 6 Marks

Task 2 – From Requirement to Hierarchy (University People) ..... 8 Marks

Task 3 – Constructors in Derived Classes & Member Classes ..... 9 Marks

Task 4 – Pointers to Derived Classes: Forensics ..... 7 Marks

Task 5 – Pointers to Objects and the this Pointer ..... 7 Marks

Task 6 – Design Challenge: Vehicle Rental Fleet ..... 13 Marks

Total: 50 Marks

TASK 1 – Class Design Review

Diagnose and Repair Duplicated Classes

1.1 Design Deficiencies

1. Repeated fields – name, id, basic and da appear in more than one class.
2. Repeated pay rule – the expression basic + basic*0.42 is duplicated.
3. Public data members – any code can corrupt the state of an object.
4. Inconsistent naming – nm/code versus name/id for the same information.
5. Unused da field – declared but never read in the pay calculation.
6. Magic number 0.42 – the DA rate is hard-coded and scattered.
7. Adding a new staff category requires writing an entirely new class and changing every loop that processes staff.
8. No common base – staff cannot be stored in a single heterogeneous collection and processed polymorphically.

1.2 Repaired Hierarchy

```
class Staff {                          // abstract base
protected:
    std::string name;
    int id;
    double basic;
    static constexpr double DA_RATE = 0.42;
public:
    Staff(const std::string& n, int i, double b);
    virtual double pay() const = 0;   // pure virtual
    virtual ~Staff() = default;
};
class Professor : public Staff {
    double research;
public:
    double pay() const override { return basic + basic*DA_RATE + research; }
};
class LabAssistant : public Staff {
public:
    double pay() const override { return basic + basic*DA_RATE; }
};
class GuestFaculty : public Staff {
    double hours, rate;
public:
    double pay() const override { return hours * rate; }
};
```

UML (textual): Staff <|-- Professor, Staff <|-- LabAssistant, Staff <|-- GuestFaculty. All data protected; pay() virtual.

1.3 Design Defence

A new staff category is added by writing one derived class that overrides pay(). Existing loops that iterate over Staff* continue to work unchanged. The DA rate lives in a single constant, and public data are no longer exposed.

1.4 Alternative Concept

A single Staff class with a type code and a switch inside pay() is also O(1) per call. Space is wasted on unused fields. Every new type forces a change to the switch and to every place that creates a Staff, violating the Open-Closed Principle. The hierarchy localises change to one new class file.

TASK 2 – From Requirement to Hierarchy

University People Management

2.1 Relationships

- Student is-a Person (single inheritance)
- Staff is-a Person (single inheritance)
- TeachingAssistant is-a Student and is-a Staff (multiple inheritance → hybrid when combined with the Person base)
- ResearchScholar is-a Student (single / multilevel)
- Address would be has-a (composition) if stored as a member

Overall graph is hybrid inheritance because of the diamond formed by TeachingAssistant.

2.2 The Diamond Problem and Virtual Bases

Without virtual inheritance TeachingAssistant contains two separate Person sub-objects (one via Student, one via Staff). Access to name or id is then ambiguous. Declaring Student and Staff as virtual public Person causes the most-derived class to contain only one shared Person sub-object, eliminating the duplication and the ambiguity.

2.3 Class Declarations

```
class Person { protected: std::string name; int id; std::string phone;
public: Person(const std::string&, int, const std::string&); };
class Student : virtual public Person {
    std::string programme; double cgpa;
public: Student(...); };
class Staff : virtual public Person {
    std::string dept; double salary;
public: Staff(...); };
class TeachingAssistant : public Student, public Staff {
public: TeachingAssistant(...);  /* must initialise Person directly */ };
class ResearchScholar : public Student { double fellowship; ... };
```

2.4 Access Table (Person member → Student)

Member in Person	public inheritance	protected inheritance	private inheritance
public	public in Student	protected in Student	private in Student
protected	protected in Student	protected in Student	private in Student
private	inaccessible	inaccessible	inaccessible

2.5 Predict the Output (virtual bases)

My predicted output: Person Student Staff TA

Actual output: Person Student Staff TA. Person is printed only once because virtual inheritance shares a single Person sub-object. Removing virtual produces Person Student Person Staff TA (or a similar double construction) and sizeof(TeachingAssistant) increases by the size of an extra Person.

2.6 Alternative Concept – Composition

TeachingAssistant could hold a Student member and a Staff member. Space then contains two Person sub-objects. Reaching name requires an explicit path (ta.studentPart.name) and there is no polymorphic Person* that points at a TA. Virtual inheritance uses less space and preserves a single identity.

TASK 3 – Constructors in Derived Classes and Member Classes

3.1 Passing Values to Base Constructors

Initialiser lists are mandatory for base classes and for members that have no default constructor. When a base is virtual, only the most-derived class calls its constructor; intermediate classes' calls are ignored. Consequently TeachingAssistant must explicitly initialise Person.

3.2 Member Classes

Address is composed inside Person (or Student). Its constructor is invoked from the enclosing class's initialiser list: home(city, pin). Course can be added to Student in the same way.

3.3 Observed Construction / Destruction Order

Rule: virtual bases → non-virtual bases (declaration order) → member objects (declaration order) → derived-class body. Destruction is the exact reverse. Traces for Student, Staff and TeachingAssistant confirmed this order.

3.4 Predict the Output (Car / Engine)

My predicted output: Vehicle Engine Car | ~Car ~Engine ~Vehicle

Actual output matches. Base Vehicle is constructed first, then the member Engine, then the Car body. Destruction runs in reverse.

3.5 Complete the Code

```
Student(const std::string& n, int a, long long r,
        const std::string& city, int pin)
    : Person(n, a), regNo(r), home(city, pin) {}
```

Expected output confirmed. Swapping the order of initialisers produces a -Wreorder warning because members are always initialised in declaration order.

3.6 Alternative Concept – Pointer Member

Address* home = new Address(...) requires an explicit delete in the destructor, a deep-copy copy constructor and a copy-assignment operator (Rule of Three). Extra time is one heap allocation; extra space is one pointer. A member object needs none of that bookkeeping.

TASK 4 – Pointers to Derived Classes: Forensics

4.1 Defects Located

- rate() is non-virtual → static binding; SeniorAccount::rate is never called through an Account*.
- Destructor of Account is non-virtual → delete through Account* calls only ~Account(), leaking the history array.
- Account copy = SeniorAccount() slices the object; the SeniorAccount part (including the pointer) is discarded.

4.2 Evidence

AddressSanitizer reports a leak of 800 bytes (100 doubles) when the virtual destructor is missing. Adding print statements shows only ~Account running for the senior object.

4.3 Repaired Program

```
class Account {
public:
    virtual double rate() const { return 3.5; }
    virtual ~Account() = default;
};
class SeniorAccount : public Account {
    double* history = new double[100];
public:
    double rate() const override { return 4.0; }
    ~SeniorAccount() override { delete[] history; }
};
// store as Account* (or unique_ptr<Account>) never by value
```

4.4 Prevention Rules

1. Any base class intended for polymorphic deletion must have a virtual destructor.
2. Functions meant to be overridden should be declared virtual and marked override in the derived class.

4.5 Predict the Output

My predicted output: 3.5 4 3.5

Actual: 3.5 4 3.5. p->rate() is non-virtual → compile-time binding to Account::rate. p->vrate() is virtual → run-time binding to SeniorAccount::vrate. The sliced copy has only an Account part, so copy.vrate() returns 3.5.

4.6 Alternative Concept – unique_ptr

std::vector<std::unique_ptr<Account>> eliminates manual delete and the leak. Slicing can still occur if someone writes Account a = *ptr; the other two defects are already cured by virtual functions and a virtual destructor. Time and space remain O(n) for n accounts.

TASK 5 – Pointers to Objects and the this Pointer

5.1 Four Ways to Reach a Cart

```
Cart c;                    // (a) direct name
Cart* p = &c;  p->add(...); // (b) pointer
Cart& r = c;  r.add(...);   // (c) reference
Cart* h = new Cart; ...; delete h; // (d) dynamic
```

5.2 Complete the Code – Method Chaining with this

```
Cart& add(const std::string& item, double price) {
    if (n < 10) { items[n] = item; prices[n] = price; ++n; }
    return *this;                 // enable chaining
}
Cart& setDiscount(double discount) {
    this->discount = discount;    // member vs parameter
    return *this;
}
```

Expected output 324.00 confirmed. If add returned Cart by value, each call would copy the whole object and chaining would modify a temporary, leaving the original unchanged.

5.3 Array of Pointers – Highest Total

```
Cart* active[100]; int nActive = ...;
Cart* best = active[0];
for (int i = 1; i < nActive; ++i)
    if (active[i]->total() > best->total()) best = active[i];
```

5.4 Comparison of the Four Ways

- Direct name – safest for locals; lifetime is automatic.
- Pointer – may be null; needed for polymorphism and dynamic allocation.
- Reference – cannot be null, cannot reseat; preferred for function parameters.
- new/delete – explicit lifetime; must not leak; prefer smart pointers.

this is a prvalue pointer to the object on which a non-static member function was called; *this is an lvalue denoting that object.

5.5 Alternative Concept – vector of Objects

std::vector<Cart> stores objects contiguously. Finding the maximum is still O(n). When the vector reallocates, any previously taken pointers or references into it become invalid; an array of pointers does not suffer that particular problem (the pointed-to Carts live elsewhere).

TASK 6 – Design Challenge: Vehicle Rental Fleet

Stage 1 – Interpret the Problem

Entities: Bicycle, TwoWheeler, Car, ElectricCar (and future types). Common data: registration number, daily rate. Motor vehicles add fuel/charge level and service date; cars add seating capacity; electric cars add battery capacity and a per-kWh charge model. Behaviour that varies: computation of rental charge. Testable requirements:

- rent() / return() work for every concrete type through a base pointer,
- late return adds a defined surcharge,
- a new type (e.g. ElectricBus) can be added without editing existing classes.

Stage 2 – Hierarchy

Abstract Vehicle (pure virtual charge()). Bicycle inherits Vehicle. MotorVehicle inherits Vehicle and adds fuel data. TwoWheeler and Car inherit MotorVehicle. ElectricCar inherits Car (or MotorVehicle) and overrides the charge model. Inheritance is single / multilevel; the fleet is stored as Vehicle*.

Stage 3 – Critical Decisions

- charge() is pure virtual in Vehicle so every concrete class must supply it.
- destructor of Vehicle is virtual.
- Fleet container: std::vector<Vehicle*> or vector of unique_ptr<Vehicle>.
- Fuel level and service date live in MotorVehicle; seating in Car; battery in ElectricCar.

Stage 4 – Class Declarations (sketch)

```
class Vehicle {
protected:
    std::string reg; double dailyRate; bool out = false;
public:
    Vehicle(const std::string& r, double rate);
    virtual double charge(int days) const = 0;
    virtual void rent(); virtual void returnVehicle(int daysLate);
    virtual ~Vehicle() = default;
};
class Bicycle : public Vehicle {
    double charge(int days) const override { return days * dailyRate; }
};
class MotorVehicle : public Vehicle {
protected: double fuelLevel; std::string serviceDue;
    /* ... */
};
class Car : public MotorVehicle {
    int seats;
    double charge(int days) const override;
};
class ElectricCar : public Car {
    double batteryKWh;
    double charge(int days) const override;  // kWh-based
};
```

Pseudocode for renting: locate a free Vehicle* in the fleet → call rent() → record start date. Returning: compute days, call charge(days), add late fee if needed, mark free.

Stage 5 – Demonstration

The implemented program rents one of each type, returns a car two days late (surcharge applied), and finally inserts an ElectricBus that publicly inherits MotorVehicle and supplies its own charge(). No existing class file was modified. Polymorphism through Vehicle* guarantees the correct charge() is called.

Alternative Concept – Type Code + switch

A single Vehicle class with an enum type and a switch inside charge() is also O(1). Space may contain unused fields for every object. Adding ElectricBus forces edits to the enum, the switch, and every factory function—precisely the changes the hierarchical design avoids.